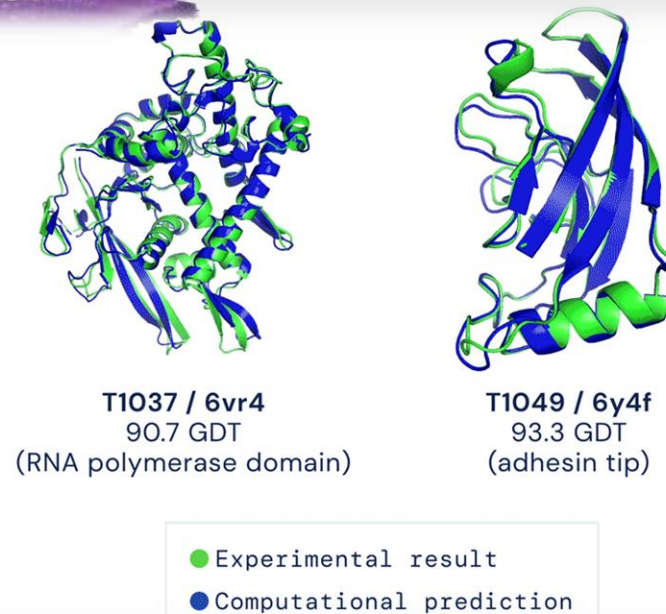


Pricing Exotics with Neural Networks

21th of jan 2021



An Exceptional Breakthrough

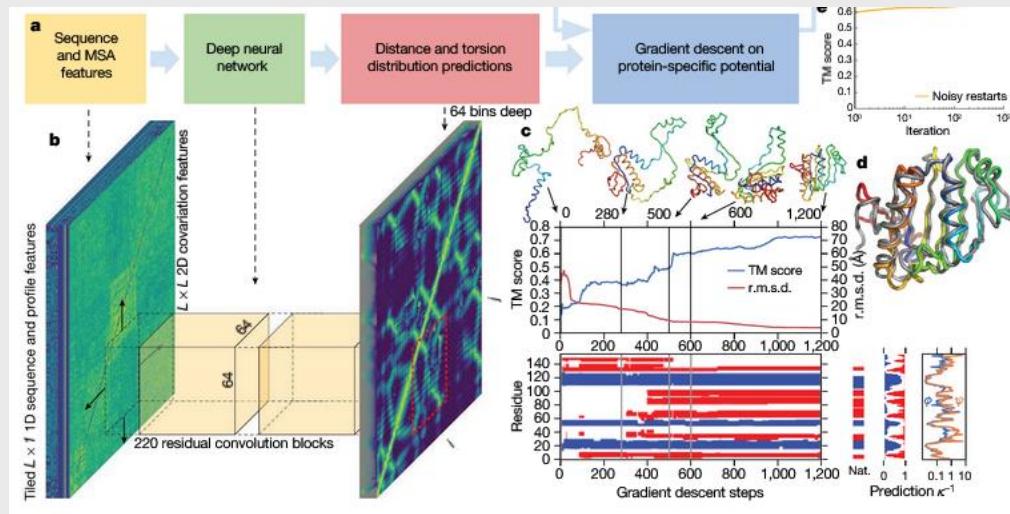
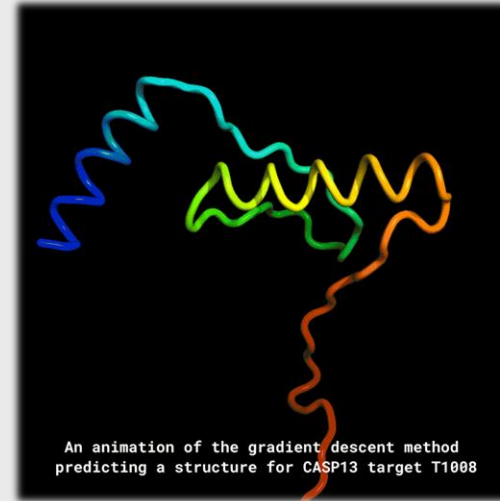


Two examples of protein targets in the free modelling category. AlphaFold (DeepMind) predicts highly accurate structures measured against experimental result.

The machinery



Deep Neural Network 223 Residual Convolution Blocks Gradient Descent



Deep Mind AlphaFold 2

What is an AutoCall Yeti-Phoenix ?



$$\begin{aligned}
 &\text{Bonus coupon} \quad \text{Yeti coupon} \quad \text{Phoenix coupon} \\
 &\text{Payoff}(t_i) = \{BO_i + \widehat{YE}_i \mathbb{1}_{\{YB_i \leq P_i\}} + (PH_i) \mathbb{1}_{\{AB_i \leq P_i\}}\} \prod_{j=1}^{i-1} \mathbb{1}_{\{P_j < AB_j\}} + \\
 &\left[\text{Gearing}(\delta_{type}(P_n - \text{Strike})^+) \left[1 - \prod_{k=0}^{n-1} \mathbb{1}_{\{DB_k < P_k\}} \right] \right] \prod_{i=1}^n \mathbb{1}_{\{P_i < AB_i\}} \\
 &\quad \quad \quad \uparrow \\
 &\quad \quad \text{Put down and in}
 \end{aligned}$$



Where

$$P_i = \text{Min}(S_1(t_i), S_2(t_i), S_3(t_i))$$

It is a complex payoff

Needs a Monte Carlo technique to be priced !



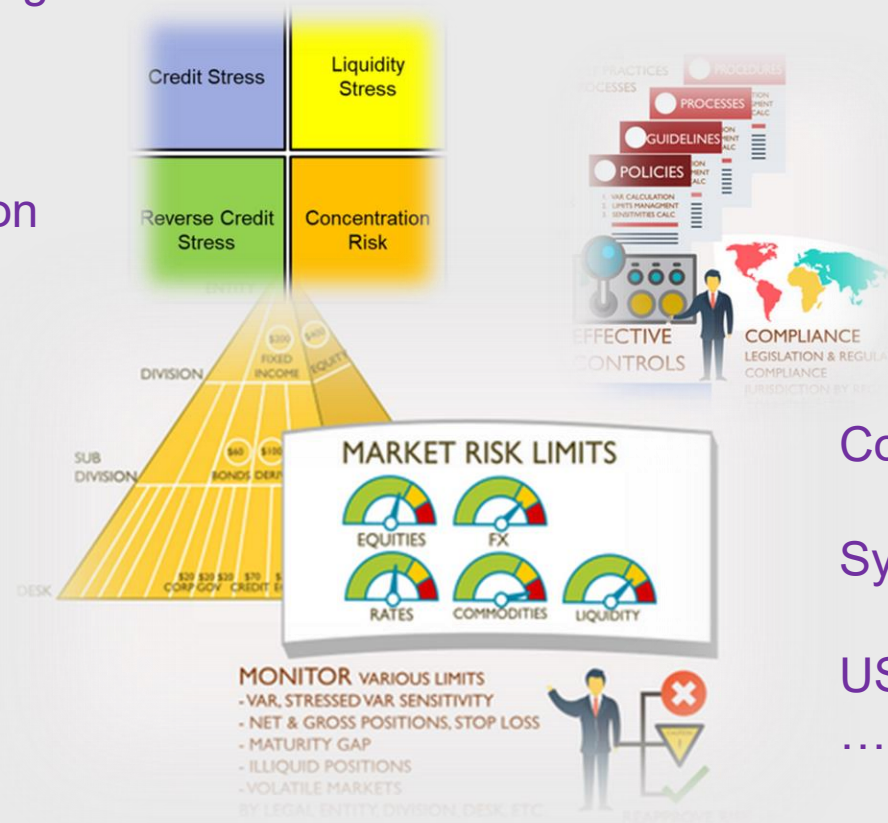
Why do we need to price so much ?

Dynamic Hedging

Risk Monitor

Capital Allocation

..



Compliance

Systemic Risk

US Regulation

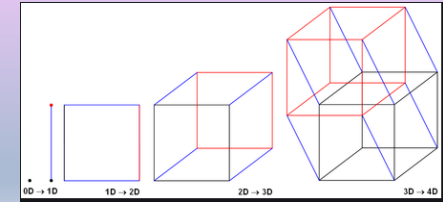
...

Why replacing a Pricer by a Neural Network is so difficult...

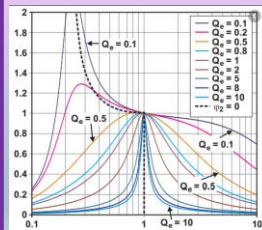
It is just a multidimensional non-linear interpolator...



The number of dimension (inputs)
can be huge : >200



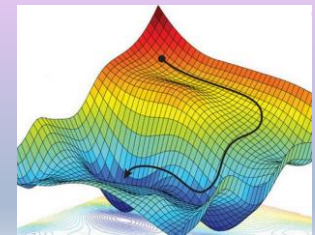
But



Pricing is a highly non linear function

The minimization of the distance
(True price – Predicted Price)

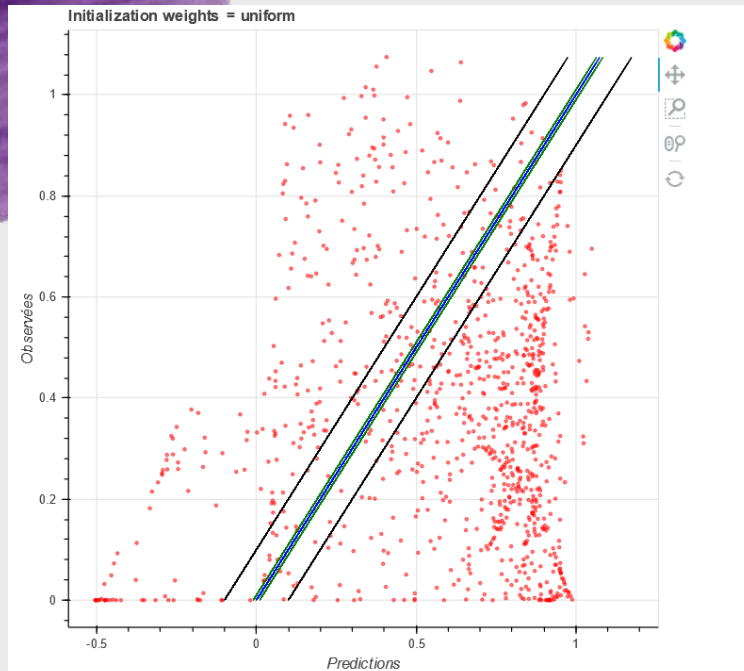
- highly non convex problem :
- a lot of local minima



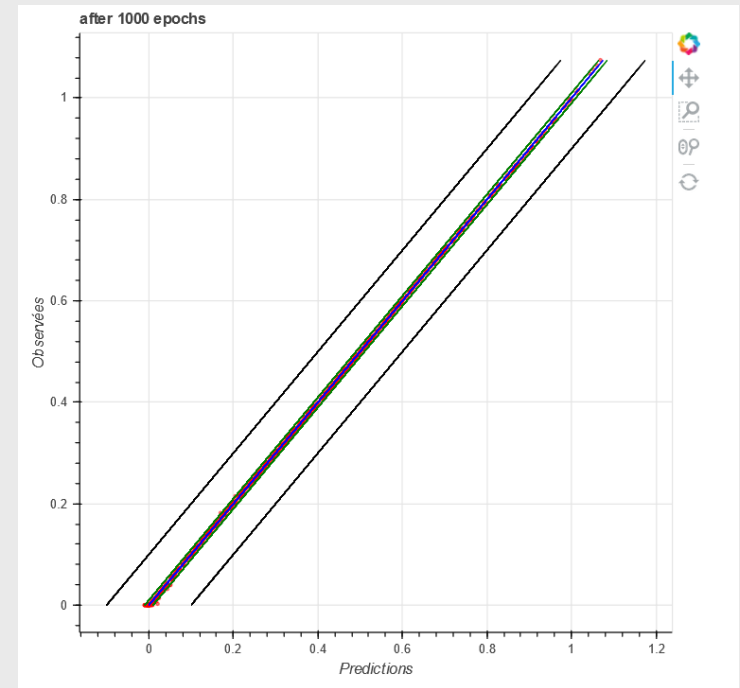
Accuracy of 0,01 %



A Graphical measure of learning succes

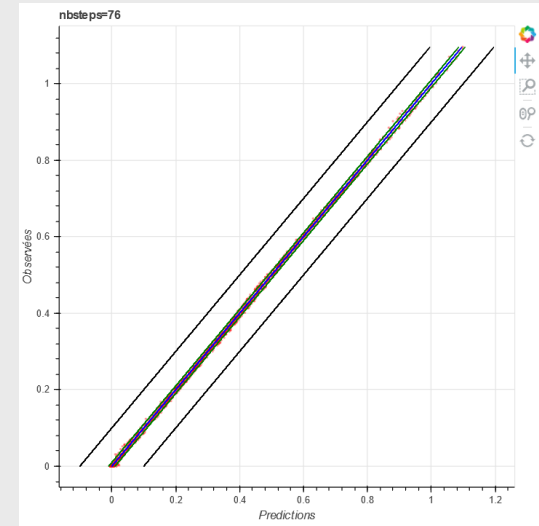
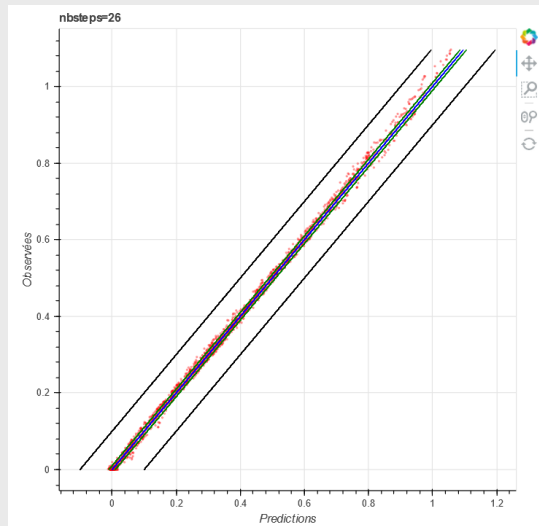
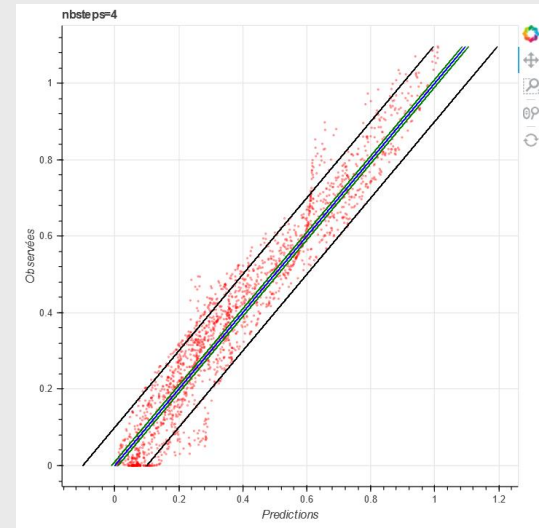
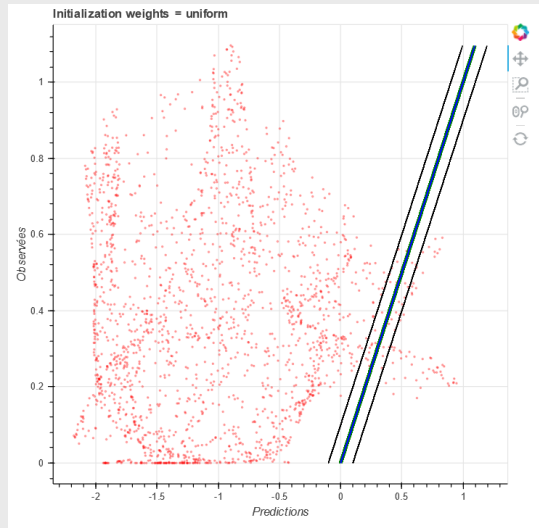


```
Entrée [332]: 1 model.evaluate(data_x, data_y)
32/32 [=====] - 0s 3ms/s
tep - loss: 0.0267 - msle: 0.0120
Out[332]: [0.026681633666157722, 0.012035036459565163]
```



```
Entrée [42]: 1 model.evaluate(data_x, data_y)
500/500 [=====] - 1s 3ms
/step - loss: 3.0196e-06 - msle: 1.6799e-06
Out[42]: [3.0195853923942195e-06, 1.6799044715298805e-06]
```

Evolution of the success graphics (BS Case : 2000 data, 2 dimensions)



6 Fast-Pricing Tricks

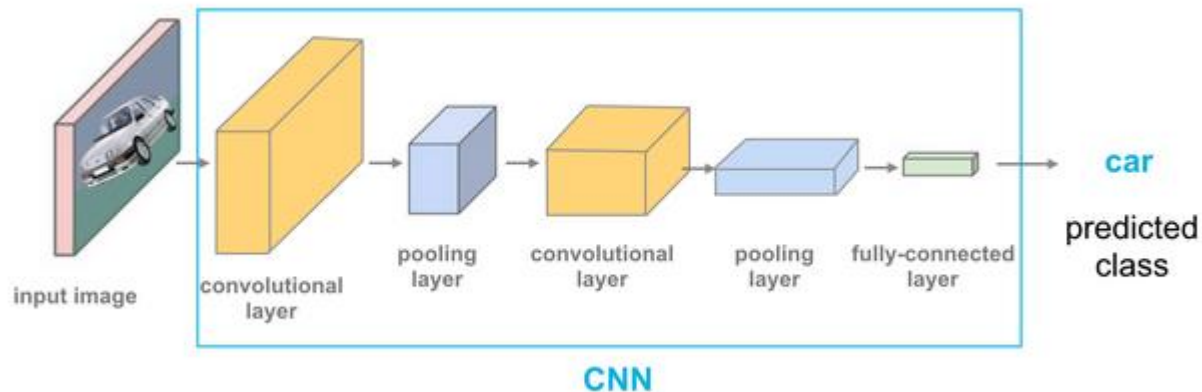


- **Convolutional First Layer**
- **Stochastic Data Phasing**
- **Learning Rate Oscillation**
- **BestChecking Oscillation**
- **Final Debiasing**
- **Genetic Programing**





- Convolutional First Layer

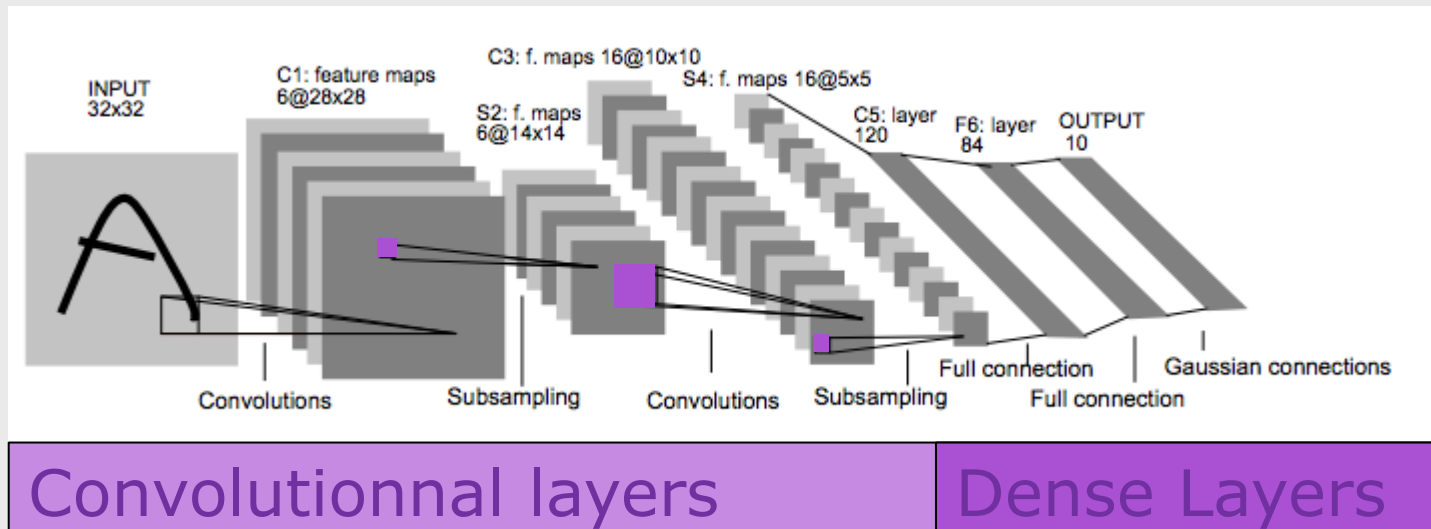


128 parametrized domains (mask)

Size of the window

```
def get_conv_model(x_train_dim, leaky_factor, drop_out):  
    model = Sequential()  
    model.add(Conv1D(128, 2, input_shape=(x_train_dim, 1), activation='relu'))  
    model.add(Flatten())  
    model.add(Dense(1024))  
    model.add(LeakyReLU(0.2))  
    model.add(BatchNormalization())  
    model.add(Dropout(drop_out))  
    model.add(Dense(512))  
    model.add(LeakyReLU(leaky_factor))
```

Convolutional Layers



Convolution:

$$f * g(x) = \int f(z - x)g(z)dz$$

$$g_A(z) = 1_A(z)$$

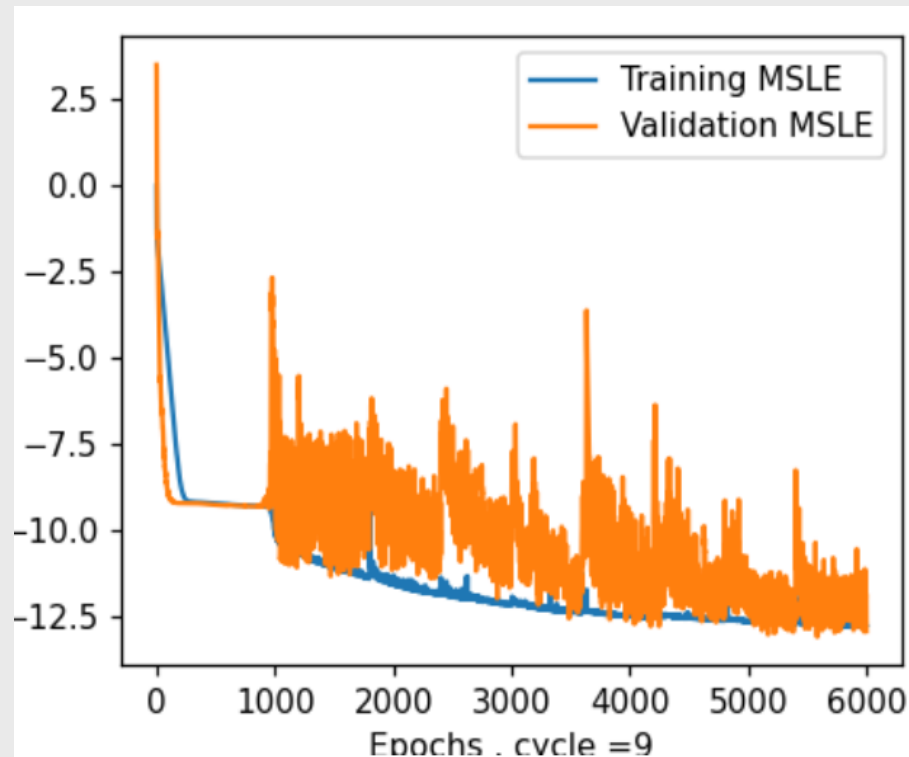
Very Common Kernel

Convolutional kernel



- **Learning Rate Oscillation**

Log(Quadratic error)



```
rlrp = ReduceLROnPlateau(monitor='val_loss', factor=factor, patience=patience, \
                          min_delta=1E-7,min_lr = 1E-9,cool_down = cool_down)
```

Stochastic Data Phasing

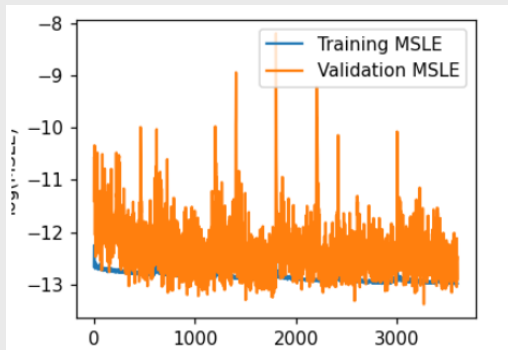


```
test_size = 0.3
```

```
x_train, x_test, y_train, y_test = train_test_split(x_values, y_values, test_size=test_size)
```

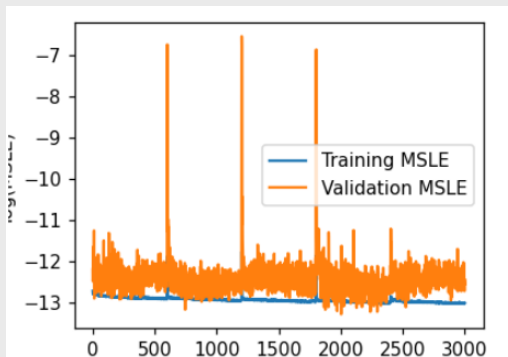
```
Entrée [24]: 1 print(bestmodel.evaluate(x_values_conv, y_values, batch_size=128))
```

1480/1480 [=====] - 24s 16ms/step - loss: 6.1781e-06 - mae: 0.0018 - mse: 6.1781e-06
[6.178058594787772e-06, 0.001808410025011036, 6.178058559503676e-06]



```
Entrée [52]: 1 print(bestmodel.evaluate(x_values_conv, y_values, batch_size=128))
```

1834/1834 [=====] - 29s 16ms/step - loss: 4.3684e-06 - mae: 0.0015 - mse: 4.3684e-06
[4.368403910480759e-06, 0.0015191652257770844, 4.368403922983122e-06]



```
Entrée [22]: 1 print(bestmodel.evaluate(x_values_conv, y_values, batch_size=16384))
```

20/20 [=====] - 29s 1s/step - loss: 3.7606e-06 - mae: 0.0014 - mse: 3.7606e-06
[3.760574770986289e-06, 0.001394811281427043, 3.760574763541416e-06]



Best Checking Oscillation

```
Epoch 01701: loss improved from 0.00001 to 0.00001, saving model to /home/olivier/Bureau/AI_Projets/LSV-LV/best_model/Totem_RM_dir/checkPoints_new/bestModel_checkpoint_Totem_RM_1701.hdf5
self.epoch_cycle= 2
epoch= 1700
Epoch 1702/1800
9/9 [=====] - 13s 1s/step - loss: 1.2441e-05 - mae: 0.0025 - mse: 1.2441e-05 - val_loss: 5.2000e-04 - val_mae: 0.0212 - val_mse: 5.2000e-04

Epoch 01702: loss did not improve from 0.00001
self.epoch_cycle= 2
epoch= 1701
Epoch 1703/1800
9/9 [=====] - 13s 1s/step - loss: 1.3142e-05 - mae: 0.0026 - mse: 1.3142e-05 - val_loss: 2.1169e-04 - val_mae: 0.0128 - val_mse: 2.1169e-04

Epoch 01703: loss did not improve from 0.00001
self.epoch_cycle= 2
epoch= 1702
Epoch 1704/1800
9/9 [=====] - 13s 1s/step - loss: 1.3171e-05 - mae: 0.0026 - mse: 1.3171e-05 - val_loss: 2.6279e-04 - val_mae: 0.0149 - val_mse: 2.6279e-04

Epoch 01704: loss did not improve from 0.00001
self.epoch_cycle= 2
epoch= 1703
Epoch 1705/1800
9/9 [=====] - 13s 1s/step - loss: 1.3738e-05 - mae: 0.0027 - mse: 1.3738e-05 - val_loss: 2.4220e-04 - val_mae: 0.0133 - val_mse: 2.4220e-04

Epoch 01705: loss did not improve from 0.00001
self.epoch_cycle= 2
epoch= 1704
Epoch 1706/1800
9/9 [=====] - 13s 1s/step - loss: 1.2927e-05 - mae: 0.0026 - mse: 1.2927e-05 - val_loss: 2.4648e-04 - val_mae: 0.0143 - val_mse: 2.4648e-04

Epoch 01706: loss did not improve from 0.00001
self.epoch_cycle= 2
epoch= 1705
Epoch 1707/1800
9/9 [=====] - 13s 1s/step - loss: 1.3175e-05 - mae: 0.0026 - mse: 1.3175e-05 - val_loss: 2.6797e-04 - val_mae: 0.0140 - val_mse: 2.6797e-04

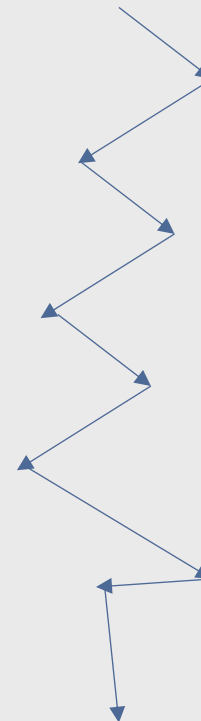
Epoch 01707: loss did not improve from 0.00001
self.epoch_cycle= 2
epoch= 1706
Epoch 1708/1800
9/9 [=====] - 13s 1s/step - loss: 1.3185e-05 - mae: 0.0026 - mse: 1.3185e-05 - val_loss: 7.4136e-05 - val_mae: 0.0074 - val_mse: 7.4136e-05

Epoch 01708: loss did not improve from 0.00001
self.epoch_cycle= 2
epoch= 1707
Epoch 1709/1800
9/9 [=====] - 13s 1s/step - loss: 1.2928e-05 - mae: 0.0026 - mse: 1.2928e-05 - val_loss: 6.2849e-05 - val_mae: 0.0066 - val_mse: 6.2849e-05

Epoch 01709: loss did not improve from 0.00001
self.epoch_cycle= 2
epoch= 1708
Epoch 1710/1800
9/9 [=====] - 13s 1s/step - loss: 1.2324e-05 - mae: 0.0025 - mse: 1.2324e-05 - val_loss: 3.6161e-05 - val_mae: 0.0048 - val_mse: 3.6161e-05

Epoch 01710: loss improved from 0.00001 to 0.00001, saving model to /home/olivier/Bureau/AI_Projets/LSV-LV/best_model/Totem_RM_dir/checkPoints_new/bestModel_checkpoint_Totem_RM_1710.hdf5
self.epoch_cycle= 2
epoch= 1709
Epoch 1711/1800
9/9 [=====] - 13s 1s/step - loss: 1.2240e-05 - mae: 0.0025 - mse: 1.2240e-05 - val_loss: 3.6161e-05 - val_mae: 0.0048 - val_mse: 3.6161e-05
```

Epoch 1700 :Improvement

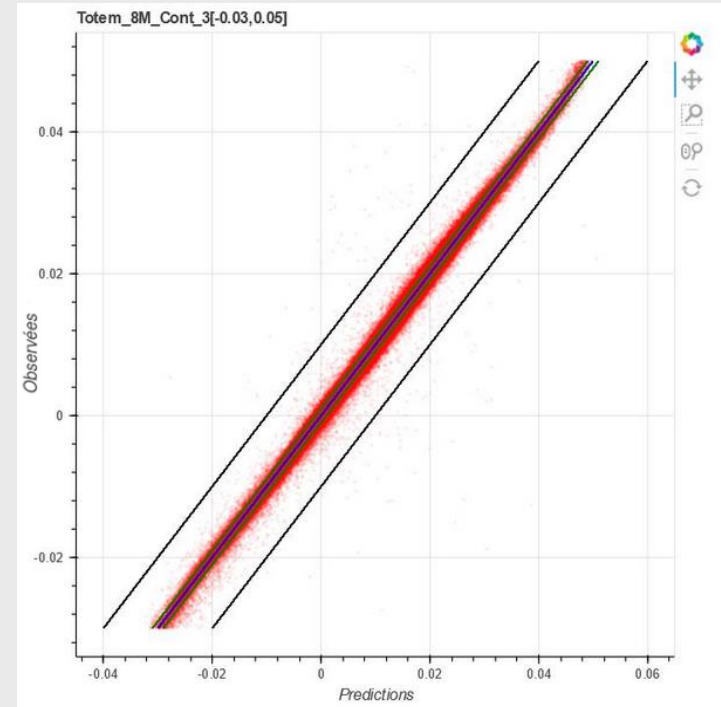
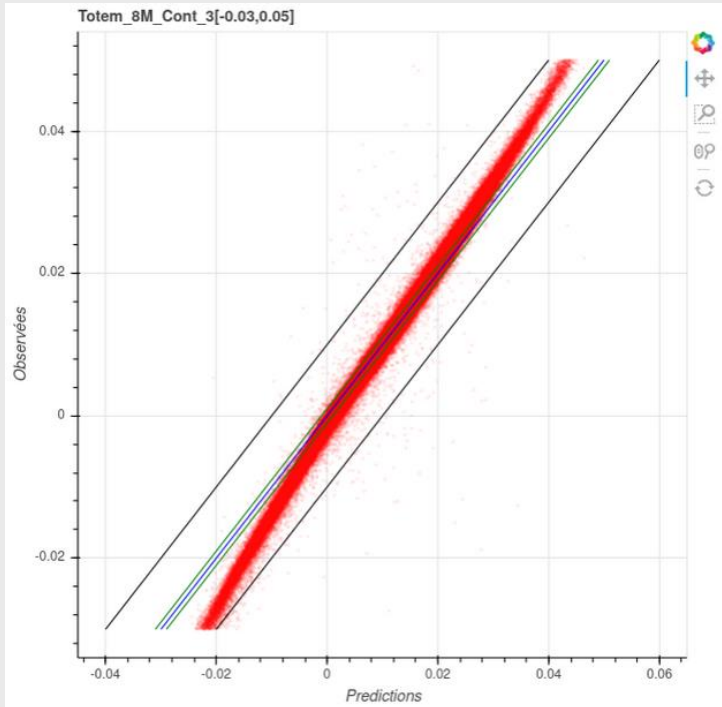


Stochastic wandering:
= Exploration

Epoch 1710 :Improvement

At the end of every phase we restart from the best !

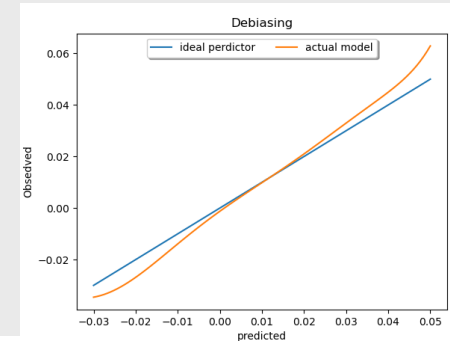
Final Debiasing



```
Entrée [18]: 1 print(bestmodel.evaluate(x_values_conv,y_values, batch_size=32768))  
14/14 [=====] - 22s 1s/step - loss: 3.4964e-06 - mae: 0.0013 - mse: 3.496  
4e-06  
[3.4963908717499152e-06, 0.0013080247305668658, 3.4963908377334723e-06]
```



```
Entrée [49]: 1 print(model3.evaluate(x_values_conv,y_values, batch_size=32768))  
14/14 [=====] - 15s 1s/step - loss: 7.4526e-07 - mae: 5.8116e-04 - mse:  
7.4518e-07  
[7.664059927517321e-07, 0.0005860571367469213, 7.658151760981224e-07]
```



Genetic Programming : Fighting the Local Minima



- How to perturb a null gradient situation without retrograding the loss

⇔ **Modify the neural network structure without increasing the loss**

- Add more neurons on layers
- Add foreign layers using a skip connection technique

- **How to minimize the impact of combinatorial explosion**

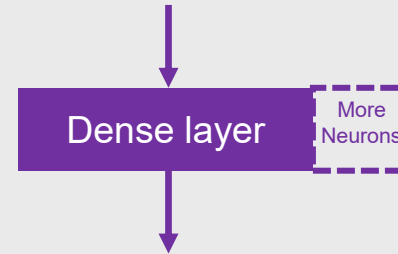
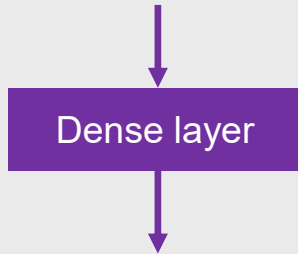
- Maintaining a population of network candidate and enforce a selection of best performing and removing the least performing asap
- Develop potential measure for selection of best candidate



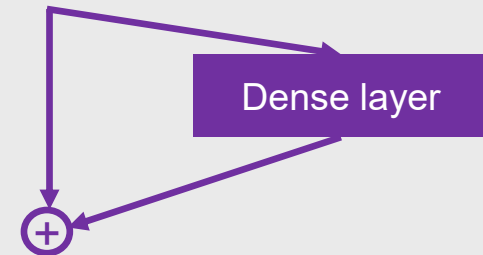


Type of Possible Mutations

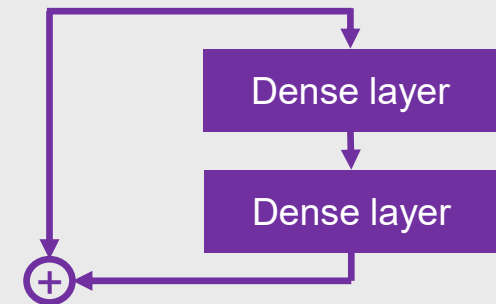
« Type 0 »



« Type 1 »



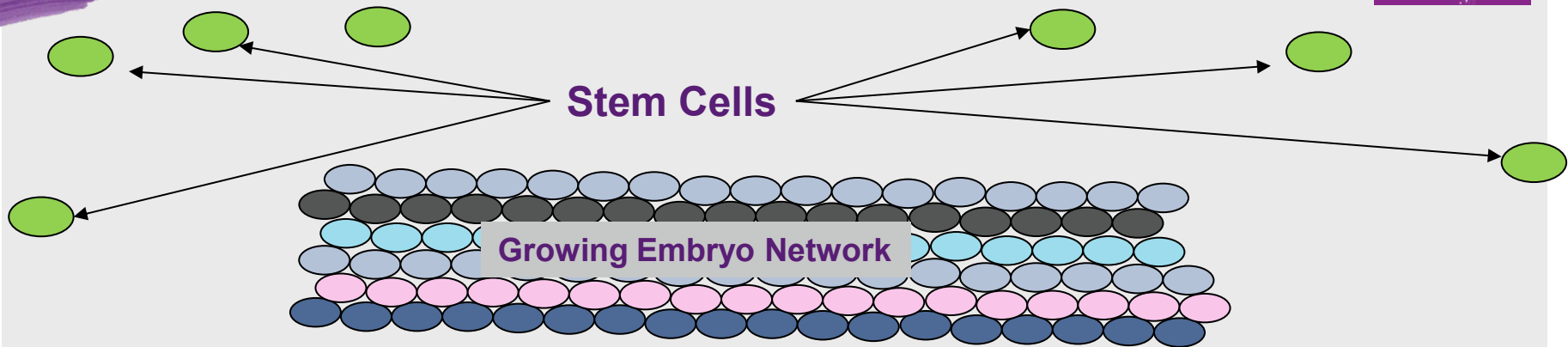
« Type 2 »



Identity
Perturbation



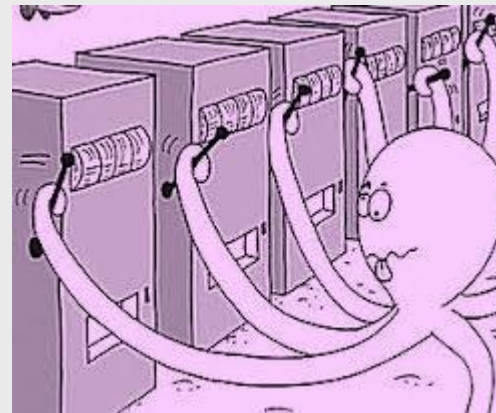
Small Neural Increment Approximation



$$Q\left\{N + \sum_{i=1}^n b_i\right\} \approx Q\{N\} + \sum_{i=1}^n (Q\{N + b_i\} - Q\{N + b_{i-1}\}) = \sum_{i=1}^n r_i$$

Multi-armed bandit

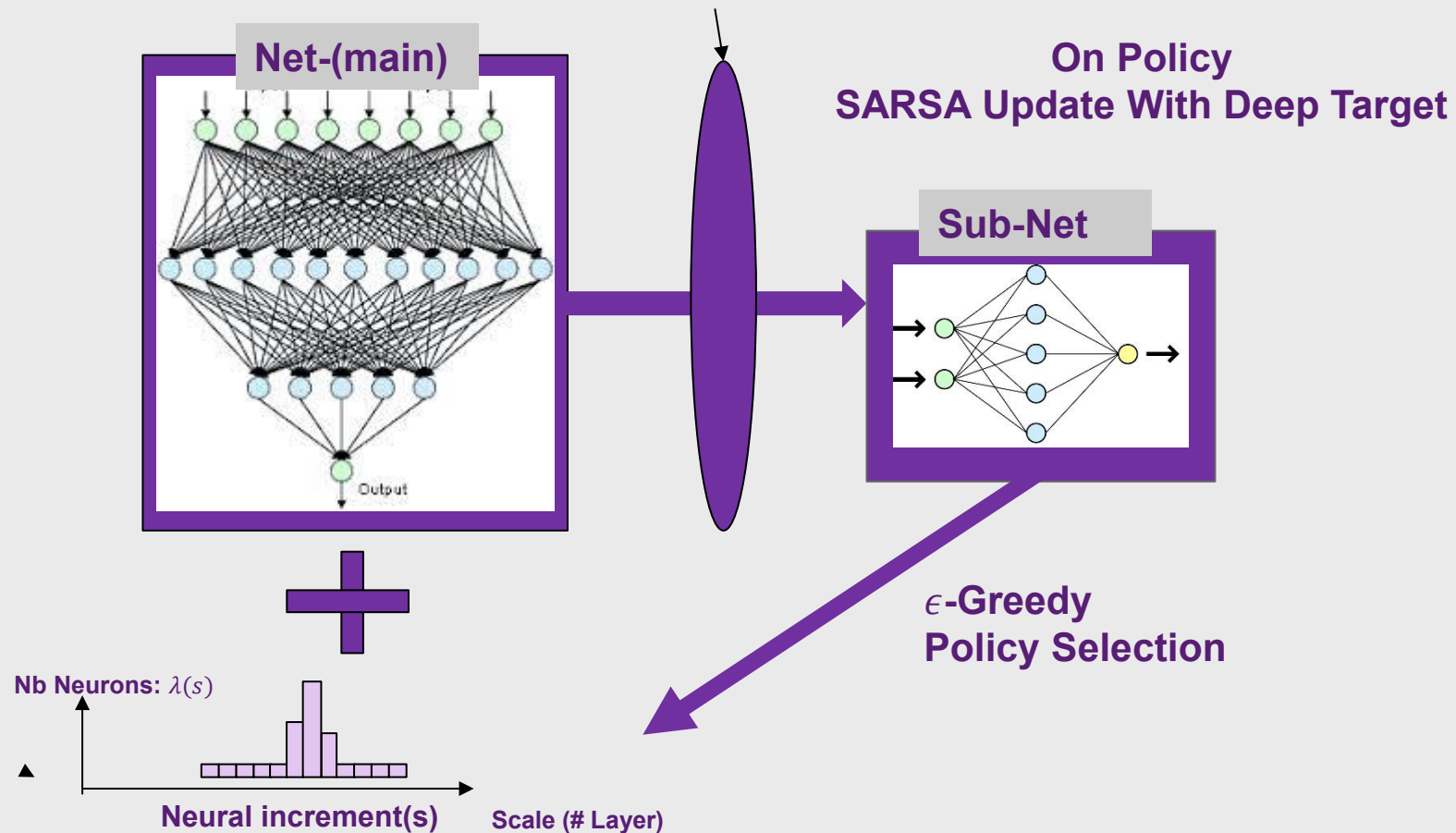
r_i Accuracy increase at step i



Deep Learning with deep TD-Learning



Few Thousand Iterations



Small Neural Increment Approximation



Policy	Return
Chosen Scale	Performance Increase
1	0,00
5	0,001
7	0,005
9	0,001
10	0,00
4	0,00
8	0,003

State	Policy	Return
Weights: Complete State	Chosen Scale	Performance Increase
W1,..Wn	1	0,00
W1,..Wn	5	0,001
W1,..Wn	7	0,005
W1,..Wn	9	0,001
W1,..Wn	10	0,00
W1,..Wn	4	0,00
W1,..Wn	8	0,003

State	Policy	Return
performance	Chosen Scale	Performance Increase
0,93	1	0,00
0,94	5	0,001
0,941	7	0,005
0,946	9	0,001
0,946	10	0,00
0,946	4	0,00
0,949	8	0,003

Multi-Armed Bandit

Full-Scale Reinforcement

Inputs = all weights

POMDP Reinforcement

Inputs = only performance

Reinforcement learning can use TD(λ)-learning

Q-value $q(\text{state}, a(k_{scale}))$: Expected value for the accuracy after learning with the bubble associated with the scale k_{scale} .

$Q(\text{state}, k_{scale}, w)$: Deep action out Q-value (deep neural network)

Epsilon-Greedy Policy Selection



With Probability ϵ

Exploration

Try a New type
of
Mutation



With Probability $1-\epsilon$

Exploitation

Greedy
Policy

Concentrate
On the Best
Mutation

Two Learning Types of Process



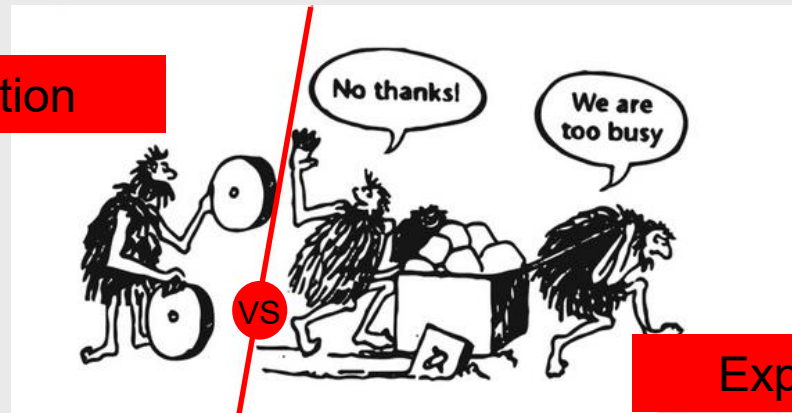
Stochastic Data Phasing

Learning Rate Oscillation

BestChecking Oscillation

Genetic Evolution

Exploration



Exploitation

Iterations of back-propagation

Improvement due to genetic programming



5% of the errors are greater than :

Model	L1 Error	Quantile 5%, bp	Quantile 1%,bp
FirstCalibr	0.0105	3.46	6.81
Genetic	0.0104	3.39	6.43
CatBoost	0.0465	13.03	21.7

gradient boosting on decision trees

Results are in % of the notional (notional = 100)



Conclusion



Multi-dimensional non-linear interpolation is possible with a precision Of 0,05 % with just a few million prices

Nb Dims	Dataset	Error (L2)	Learning Effort
23	1 M	$6.8 \cdot 10^{-8}$	4 GPU Maxwell 3 days
191	7 M	$3.5 \cdot 10^{-7}$	16 GPU V100 3 Days
61	1 M	$7.6 \cdot 10^{-7}$	4 GPU Maxwell 3 Days

Harmonious usage of exploration and exploitation to address local minima and slowness of convergence

